Project Design Phase — Solution Architecture

Date	8 July 2026
Team ID	SmartBridge Externship
Project Name	FinRelief AI
Maximum Marks	4 Marks

Architectural Approach

FinRelief AI is built as two fully decoupled services that communicate only through a REST API: a React single-page frontend and a FastAPI backend. The backend never renders HTML — every response is JSON — which allows the two sides to be deployed and scaled independently. In production, the frontend runs on Vercel and the backend runs on Render.

Request Lifecycle (Example: Generating a Negotiation Letter)

1. The React frontend sends `POST /letters/{loan_id}` via Axios, with the JWT attached in the `Authorization` header.
2. FastAPI verifies the JWT and validates the request through Pydantic before any business logic runs.
3. The letters router pulls the loan's stored financial data and calls the Gemini client to draft the letter body.
4. If the Gemini API key is valid and the call succeeds, the letter is generated and tagged as AI-sourced.
5. If the Gemini call fails — timeout, non-2xx response, or an unparseable response — the backend automatically falls back to a static professional template instead, and the letter is tagged accordingly.
6. The letter is saved through SQLAlchemy into Neon PostgreSQL, and the JSON response is returned to the frontend for display or PDF download.

Database Design

The schema is organised around four related entities:

- **Users** — account details, hashed passwords, and authentication metadata
- **Loans** — the borrower's loan details: lender, amount, EMI, overdue days, monthly income
- **Settlement Snapshots** — a record of each settlement calculation run against a loan, including DTI ratio and stress level, used to power the dashboard's trend charts over time
- **Negotiation Letters** — the generated letter body, along with whether it came from Gemini or the fallback template, and a version history per loan

---

Why Neon PostgreSQL Instead of a Local Database

Render and Vercel both use ephemeral filesystems — anything written to local disk is wiped on every redeploy. A file-based database sitting in that environment would lose all user data the next time the app redeploys. Neon, hosted independently of the application's own deploy lifecycle, persists regardless of how often the backend is redeployed. SQLite is used only for the isolated automated test suite, where a disposable, file-based database is the right tool for a clean slate on every test run.

Application Characteristics

Scalability — The frontend and backend are deployed as separate services and can be scaled independently. Because the backend is stateless (JWT-based, no server-side session storage), multiple backend instances can run behind a load balancer without shared session state to coordinate.

Security — Passwords are hashed with bcrypt before storage; sessions are handled through signed JWTs; secrets are loaded from environment variables and never committed to source control; SlowAPI rate-limits sensitive endpoints against abuse.

Availability — The Gemini-with-fallback pattern means the negotiation letter feature keeps working even during an AI service outage.

Performance — The settlement calculation is a pure, synchronous function with no external calls and returns near-instantly; the only latency-sensitive step in the system is the Gemini call during letter generation, which is time-boxed so a slow or failed AI response cannot hang the request indefinitely.